SENG 401

Design Artifact

Group 10

Ali Awan

Sahil Bacchus

Ali Chaudhary

Tyseer Shahriar

Jibran Somroo

Yahya Syed

2026-03-27

Architecture Overview:

The Climate Accord game is structured into 3 main categories: the user interface, the backend, and the data management layer. The user interface is implemented in the src and public folders using a web-based frontend, more specifically TypeScript. This component is responsible for handling what the user sees and controls, such as the login/signup screens, hosting/joining games, and navigating through the screens. It sends user inputs to the backend and updates the game based on the responses received.

The backend, which is in the server folder, is responsible for handling the core functionality of the game. It is split into multiple modules that handle different aspects of the game. The game module handles core gameplay such as bill generation, voting, next phase and the lobby handles the joining, kicking, and hosting logic. The state.ts file is responsible for maintaining and updating the current game state across all players ensuring all games are in sync.

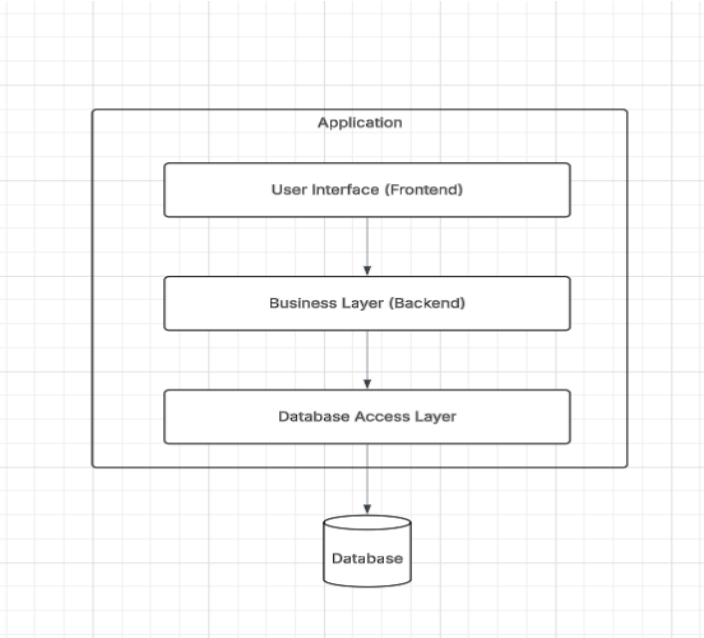
The data management layer is implemented within the server directory using Supabase and is accessed through the db.ts module. This component is responsible for handling persistent data operations related to game sessions and player participation. The db.ts file defines a database interface that allows the backend to create new game sessions, add players to games, and record game outcomes such as the winning faction.

Architecture Style: Layered Architecture

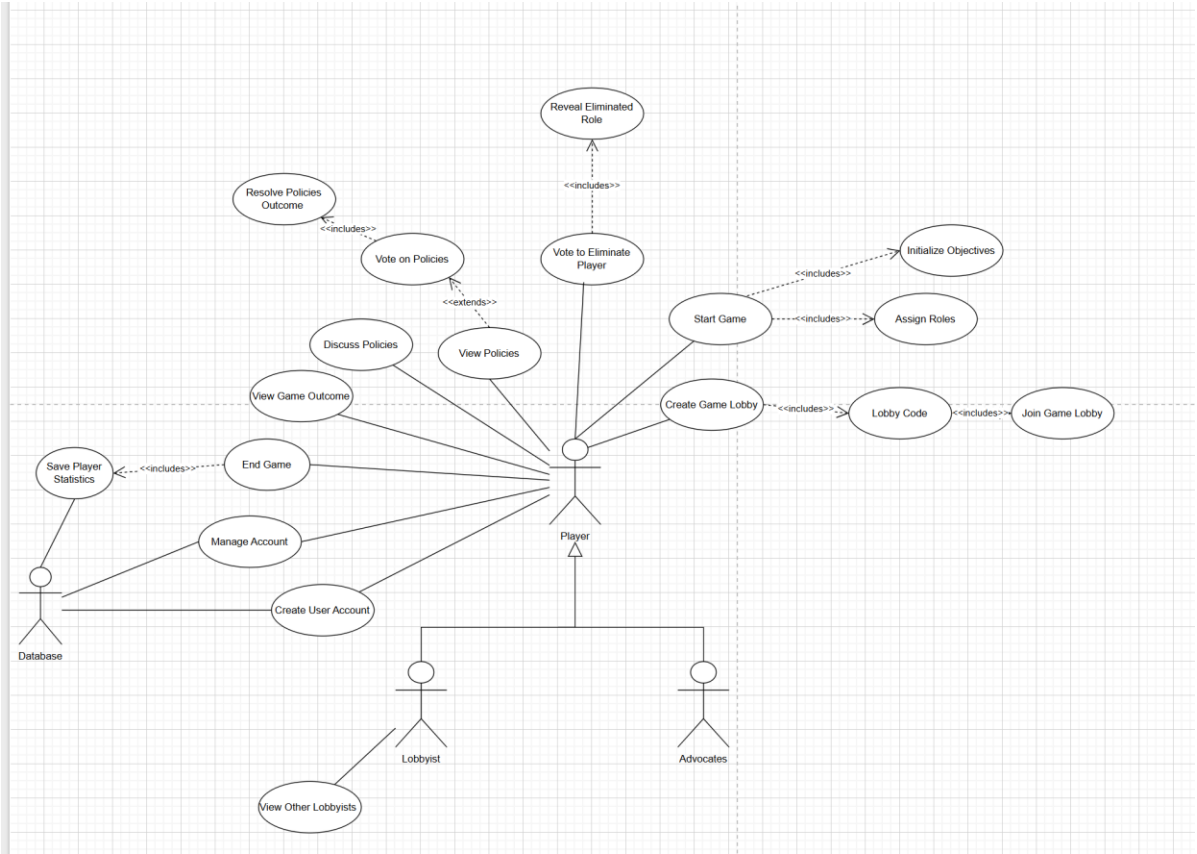
The system is primarily designed using a layered architecture**,** separating the user interface, backend game logic, and data management. This structure improves maintainability and clarity by assigning each layer a distinct responsibility, allowing changes to be made with minimal impact on other components. This approach was chosen because it promotes modularity and simplifies development and debugging. However, the architecture is not strictly enforced in all cases. Since Supabase provides Backend-as-a-Service (BaaS) features, some operations such as user authentication (login and sign-up) are handled directly from the frontend using the Supabase client. This deviation was intentional to reduce development complexity and take advantage of built-in authentication features, while core game logic and data handling remain within the backend, preserving the overall benefits of the layered architecture.

Diagrams

Architecture Diagram:



Use Case Diagram:



Generative AI Usage:
Our group used ChatGPT (OpenAI), GitHub Copilot, and Codex as tools during the development of Climate Accord. ChatGPT was used to help clarify concepts from the lecture and get styling guidance for the user interface design, helping us improve the layout and usability of the application. ChatGPT also helped us understand how to use Supabase for authentication and data management. GitHub Copilot was used for inline coding suggestions to speed up routine development tasks, such as writing basic control structures and improving code efficiency. Codex was used to support our understanding of how to properly connect the project to API services. However, all AI-generated content was reviewed and modified to ensure it accurately reflects course material. These tools were used to support our learning, rather than replacing the work that was needed to be done.